IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

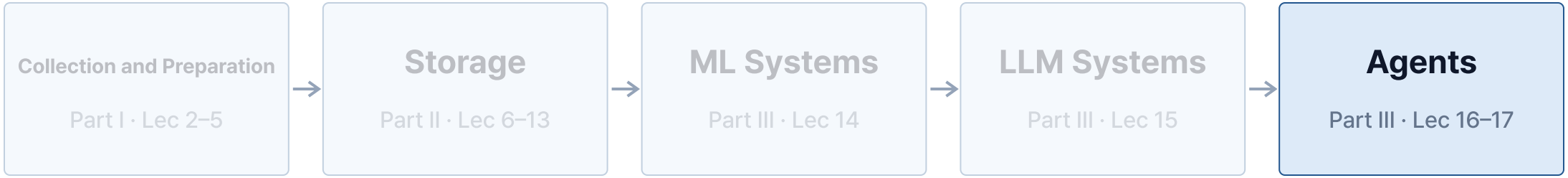
16.

Agents

Sangwoo Mo · POSTECH

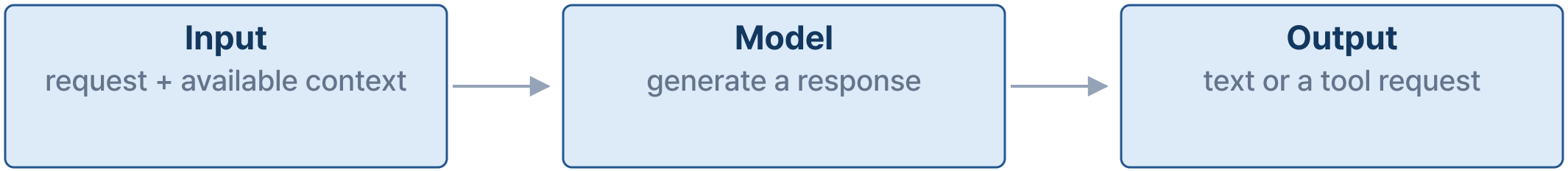
Industrial & Management Engineering

Where We Are



Part III connects stored data to predictions, answers, and actions

Recap: Model Input and Output



A tool request lets the application obtain outside evidence

Outline: Agents

01

Choosing and Running Actions

use tool results to choose the next action

02

Checking and Controlling a Run

check evidence, enforce permissions, and bound retries

03

Connecting Steps and State

route branches and preserve progress across steps

01 · Choosing and Running Actions

01

Choosing and Running Actions

use tool results to choose the next action

02

Checking and Controlling a Run

check evidence, enforce permissions, and bound retries

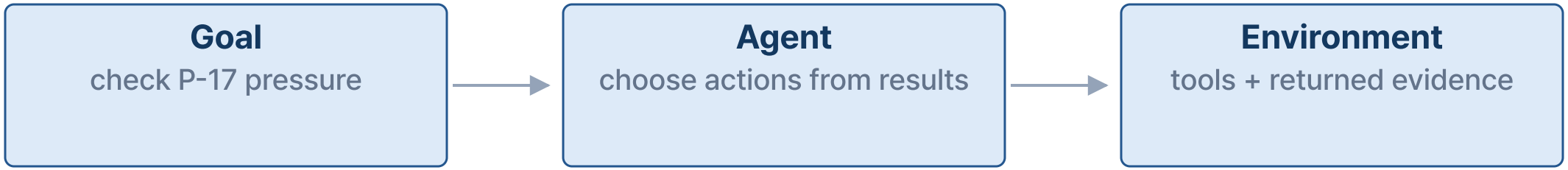
03

Connecting Steps and State

route branches and preserve progress across steps

AI Agent

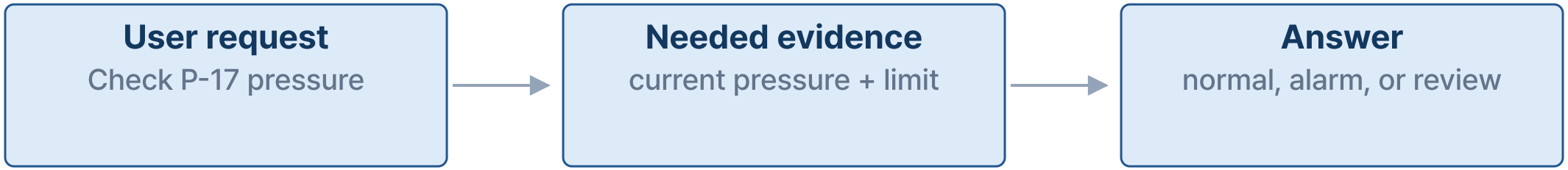
An agent uses observations to choose actions toward a goal



The goal stays fixed while the next action can change

One Inspection Request

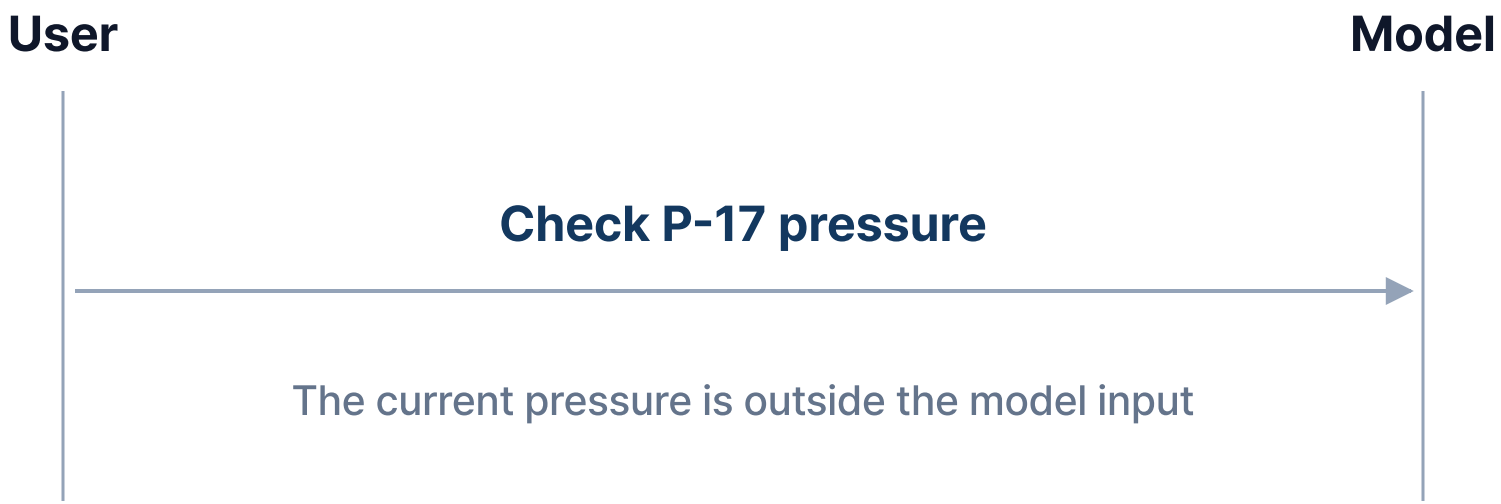
The answer needs a measurement and a comparison rule



One request connects the steps in this lecture

A Missing Measurement

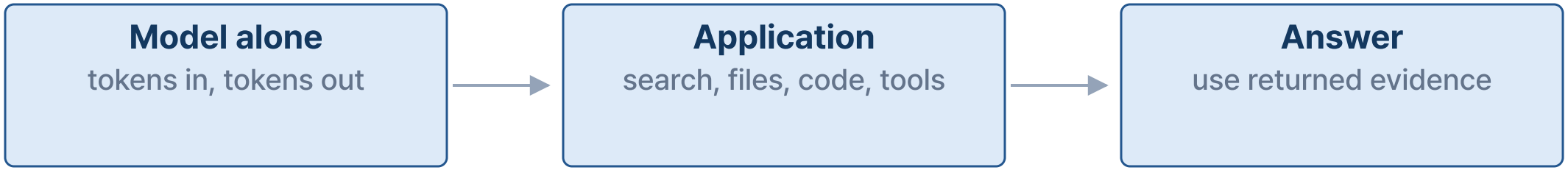
A model needs outside evidence for the current reading



A fluent answer does not supply the missing measurement

Model and Application

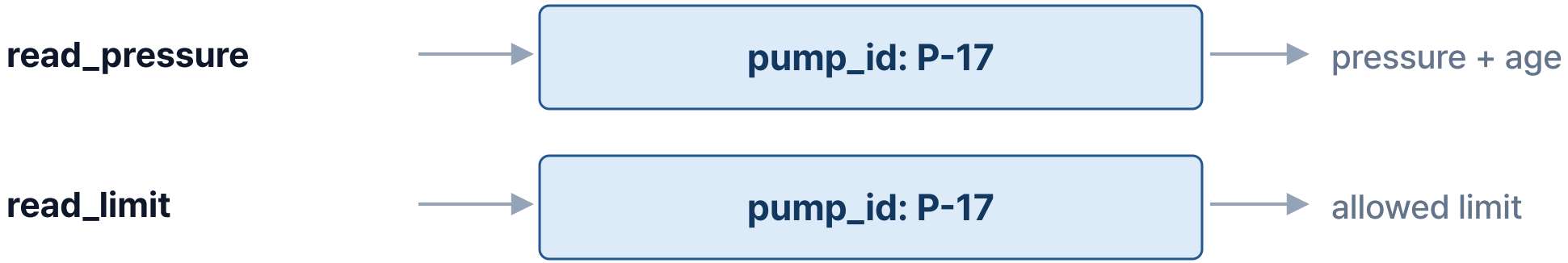
The application supplies capabilities beyond model parameters



Current measurements must enter through the application

Two Available Tools

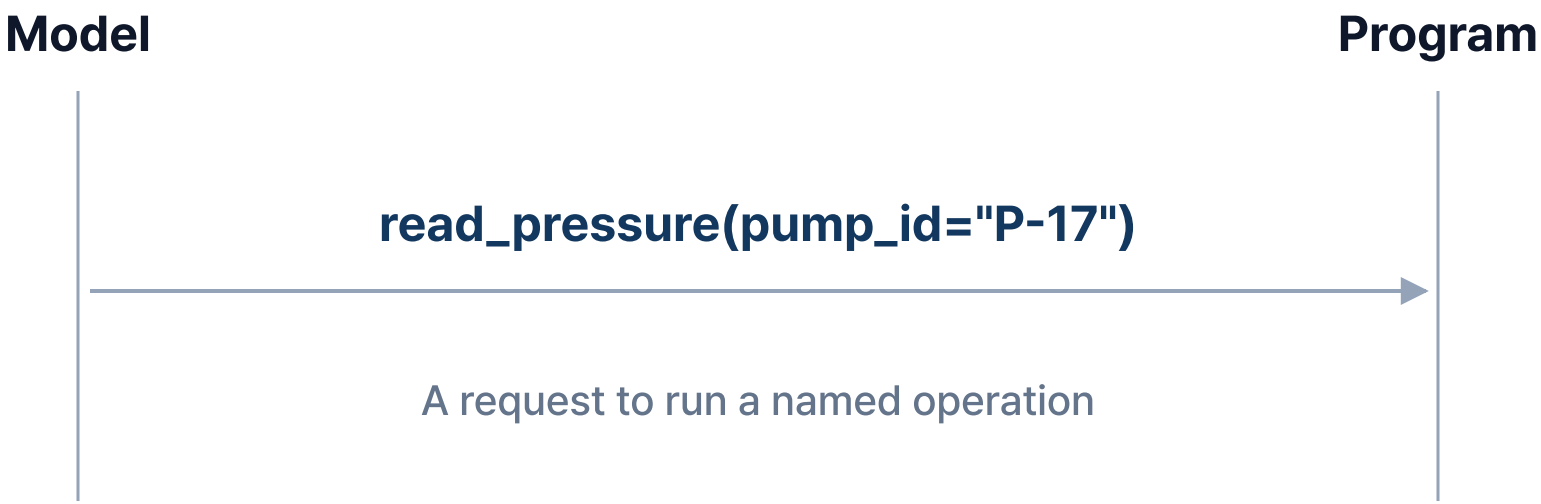
A tool names an operation that the program can run



The same pump ID selects the data for each tool

Model Tool Request

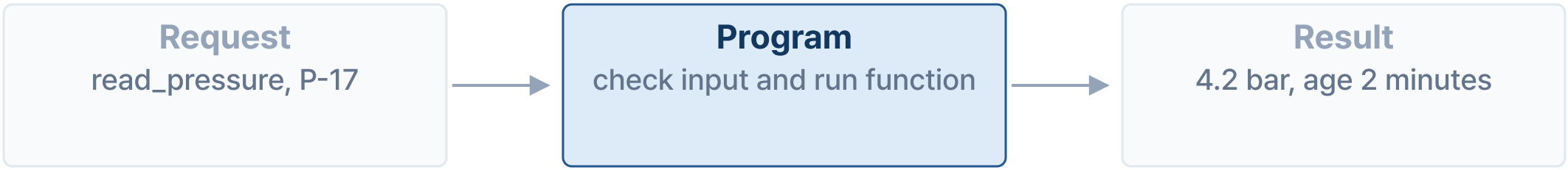
The model returns a tool name and its input



The request identifies an operation; execution has not happened yet

Program Execution

The application executes the requested operation



The reading comes from the executed function

Tool Result

The observation includes the value and how old it is



P-17 returns 4.2 bar from two minutes ago

An Input Check

The program checks the input before running the operation



A proposed input can fail before any tool executes

Demo: Run One Tool

Offline exercise: fixed data, real library execution, no model inference

1. Run the starter and locate the pressure and age

2. Change P-17 to P-99. Does the program return a reading?

```
Type code or click Starter
```

```
Run code to inspect its output.
```

Run | Starter | Reset

Ready

LangChain.js · JavaScript · Ctrl/Cmd+Enter · Real library execution with local data

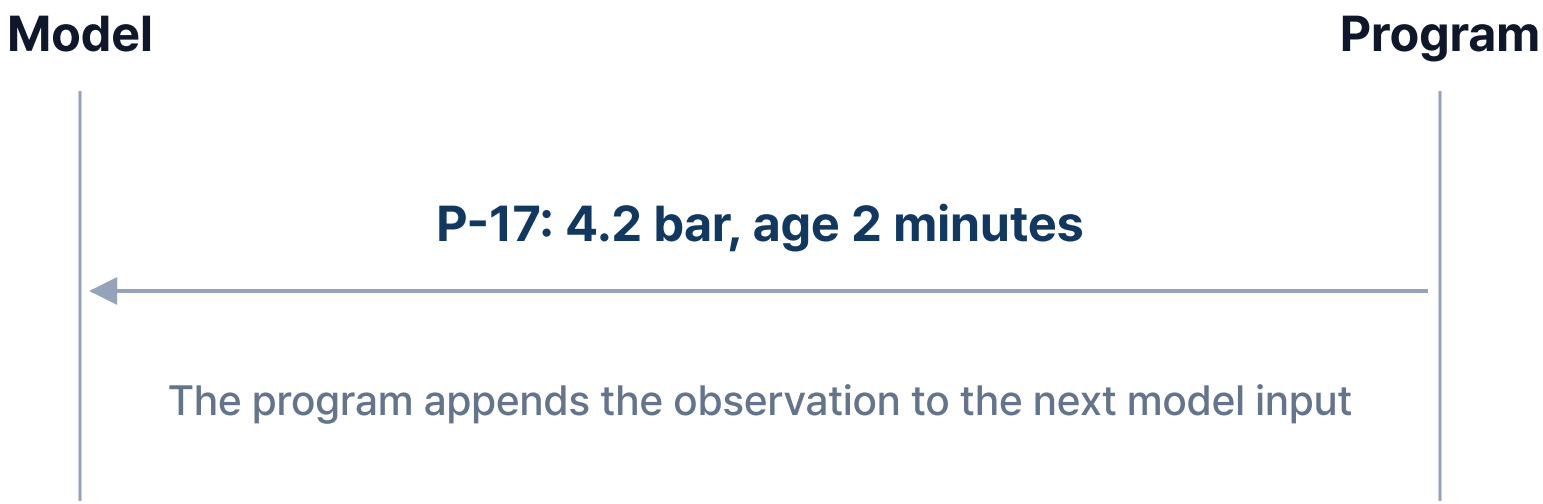
Answer: Run One Tool

The output shows what the program actually executed

```
{
  "pump_id": "P-17",
  "pressure_bar": 4.2,
  "age_minutes": 2
}
```

Returned Observation

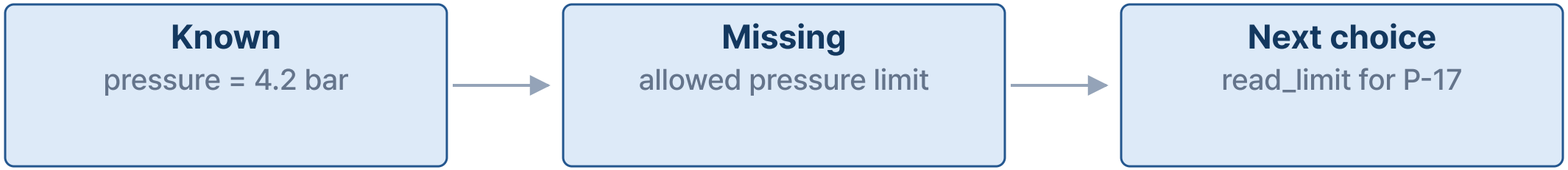
The next model call receives the executed result



The observation becomes evidence for the next decision

Missing Comparison Rule

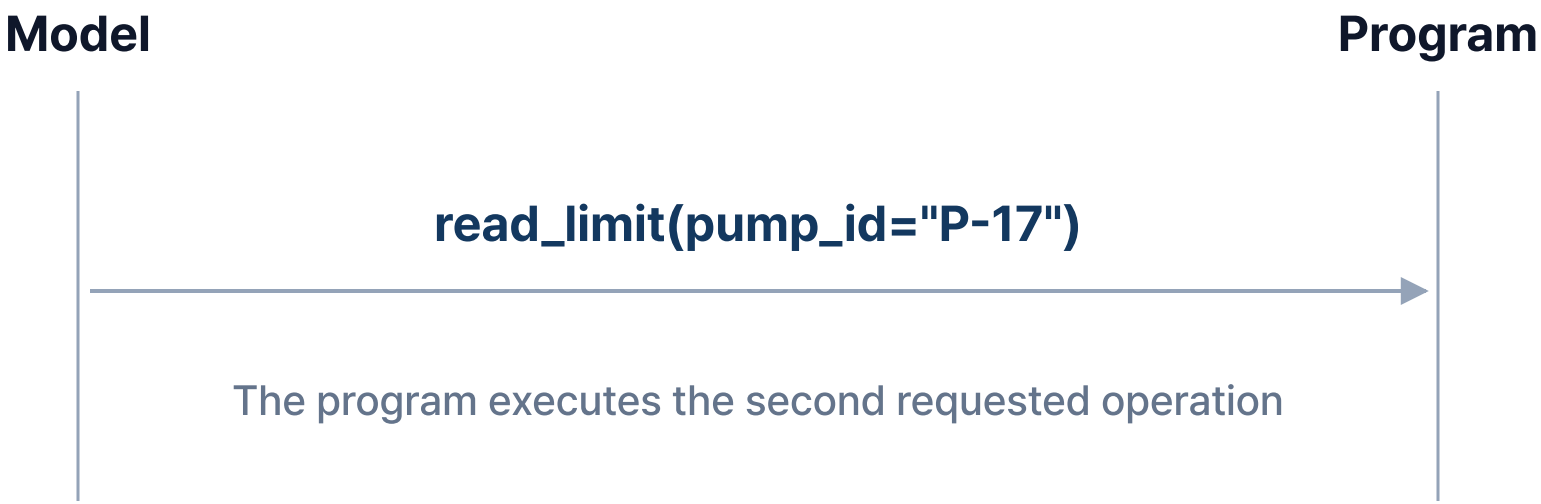
The reading alone does not determine the inspection result



The model can request the missing comparison rule

The Second Tool Call

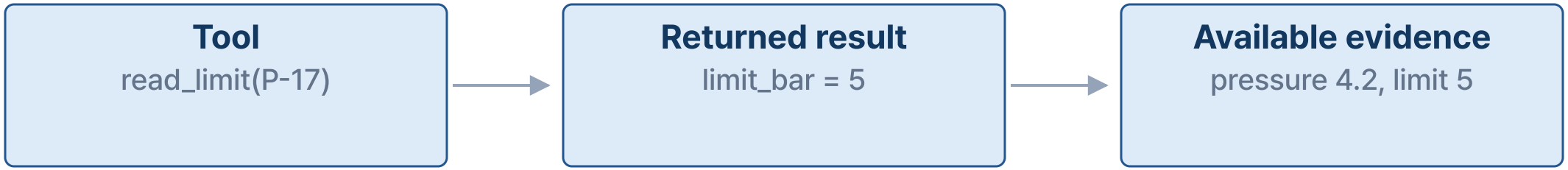
The model chooses another tool after the first result



The second request uses the same pump ID

The Comparison Rule

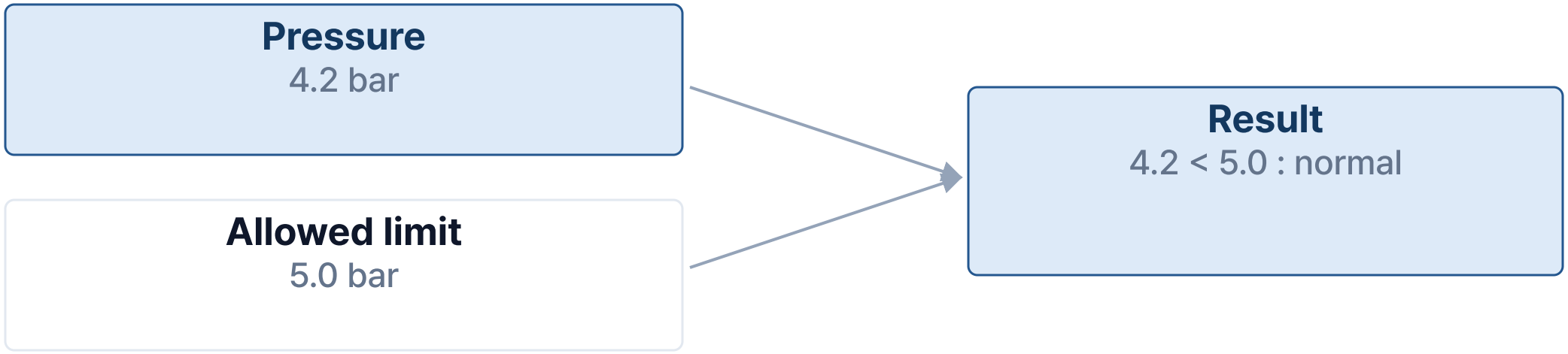
The second result completes the needed evidence



Both observations now support the comparison

Evidence and Answer

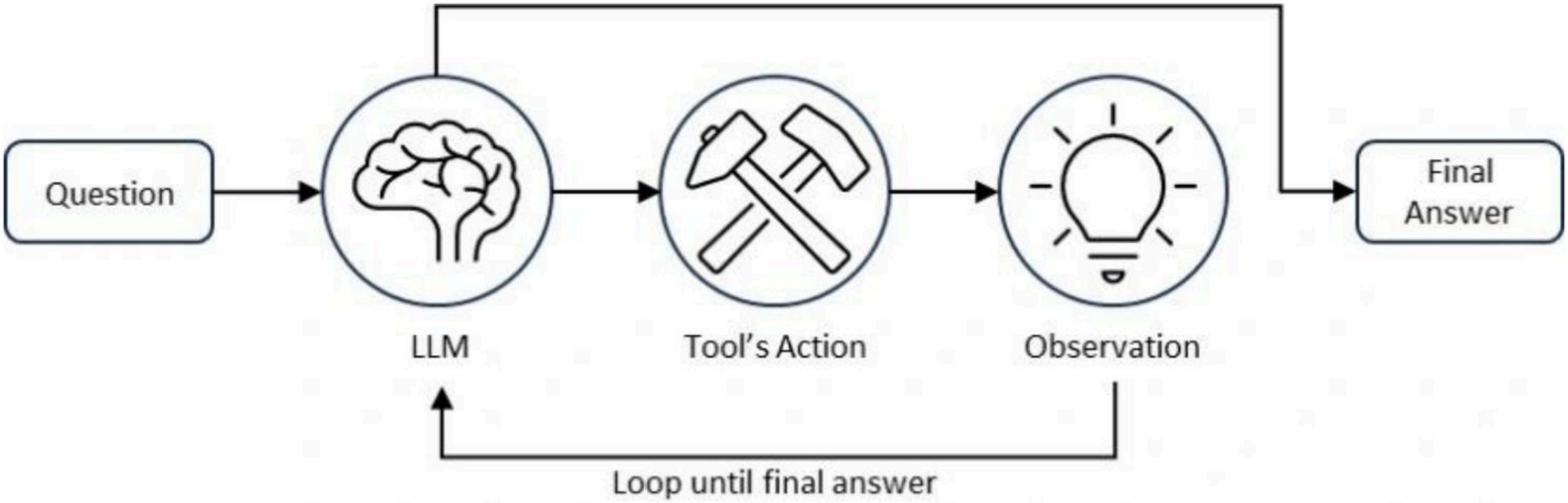
A fresh reading below the example limit gives normal



The answer uses the two returned values

The Agent Loop

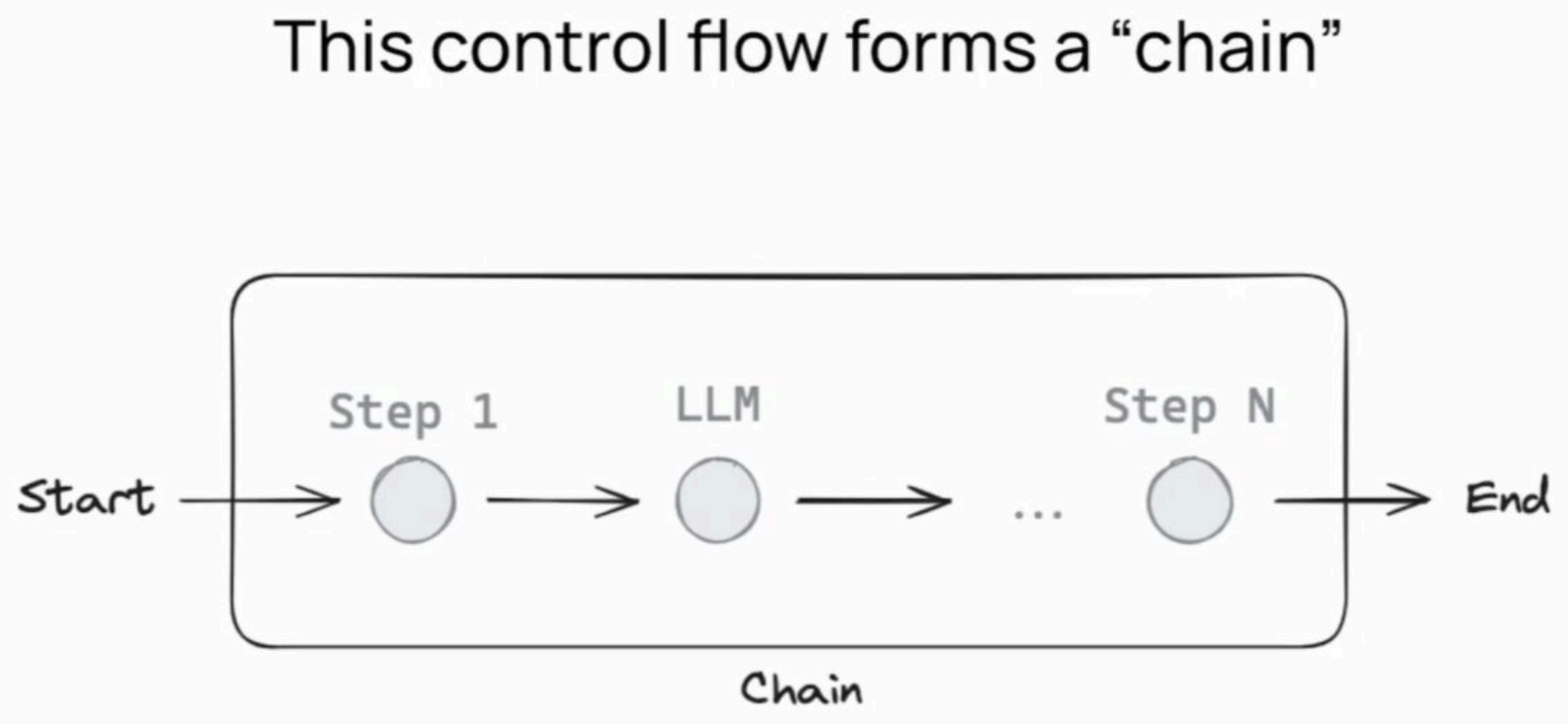
ReAct alternates model decisions, actions and observations



The returned observation changes what the model can decide next

A Fixed Chain

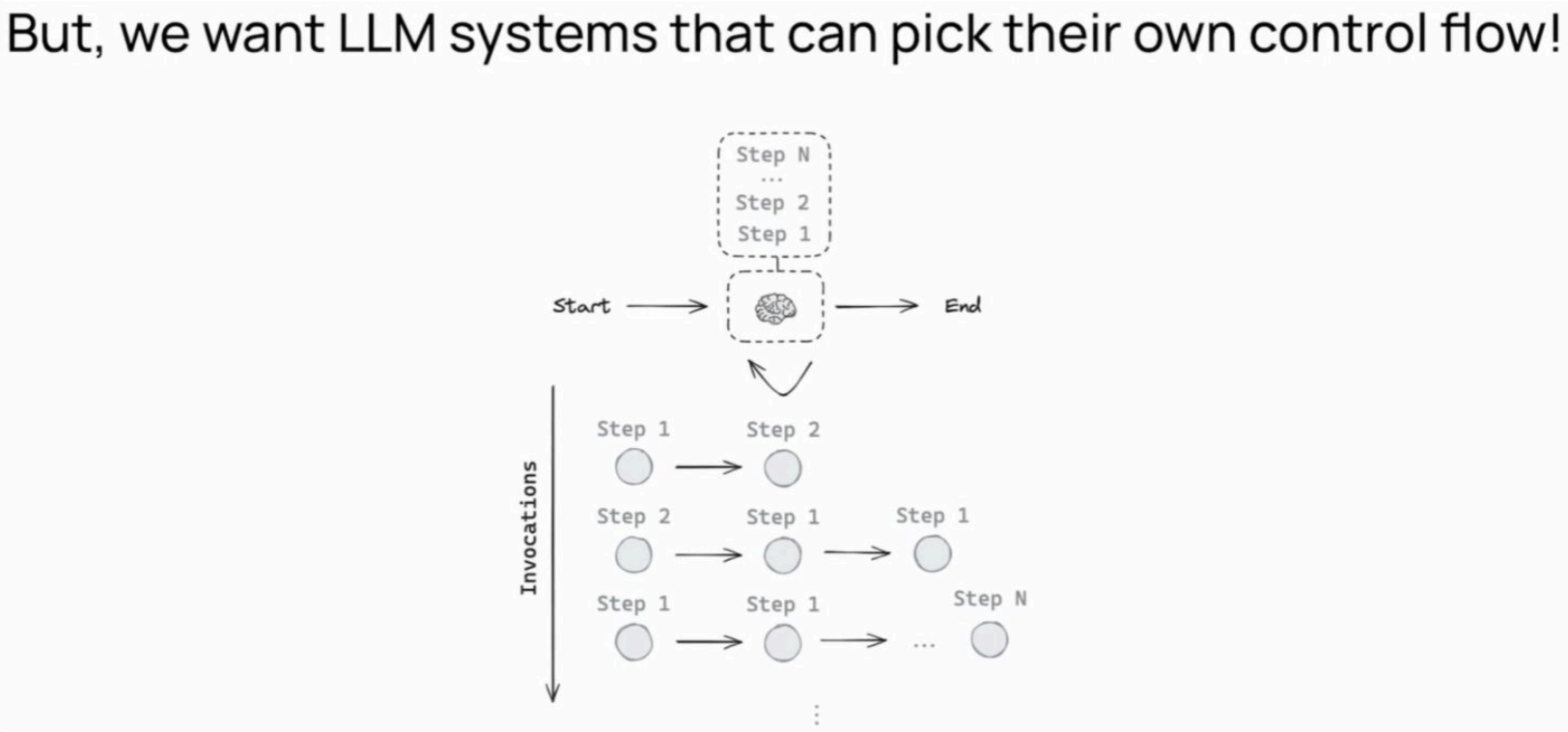
A fixed chain follows the path chosen by the application developer



The next step is fixed even when the earlier answer changes

Model-Selected Routes

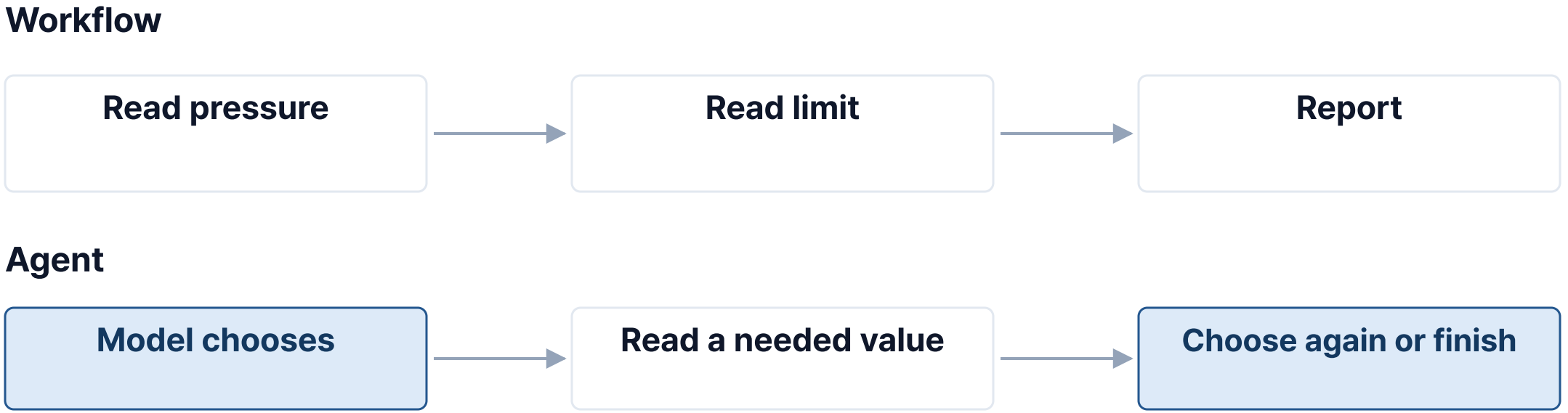
An agent can choose a route from the current observation



A chosen route still needs checks and an execution limit

Workflow and Agent

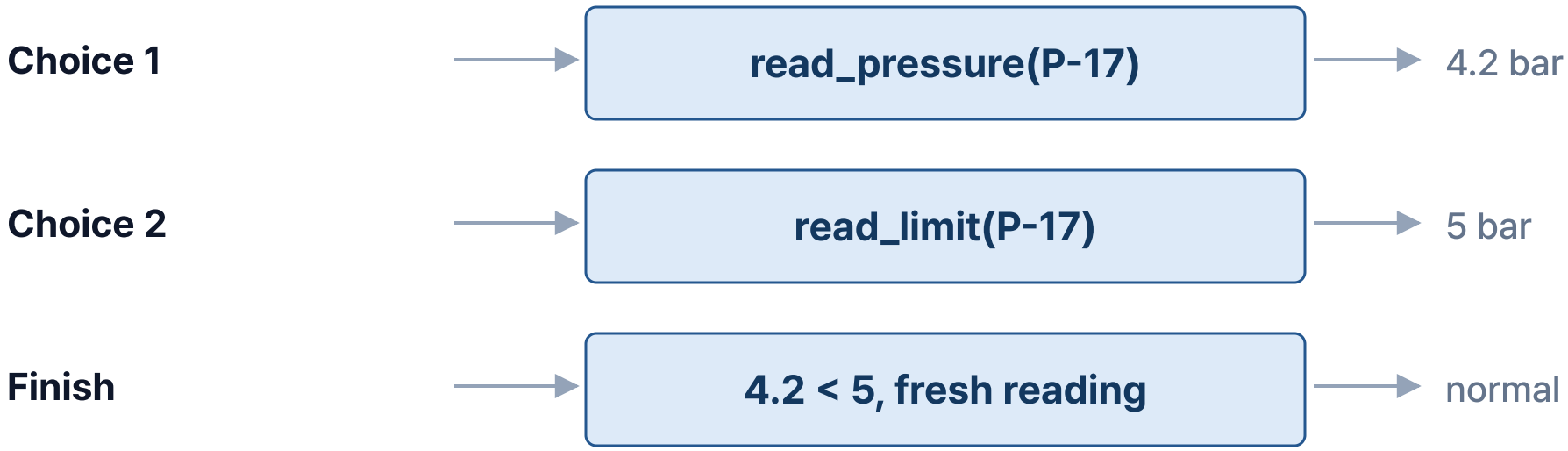
A workflow follows code-defined paths; an agent lets the model choose actions



Both approaches still use a program to execute the tools

Inspection Steps

Each result supplies evidence for the next step



Two tool results support one final answer

Demo: Follow the Agent Loop

Offline exercise: fixed data, real library execution, no model inference

1. Run the starter and match the two tool results to the final outcome
2. Change step<3 to step<2. Does the loop reach the final response?

```
Type code or click Starter
```

```
Run code to inspect its output.
```

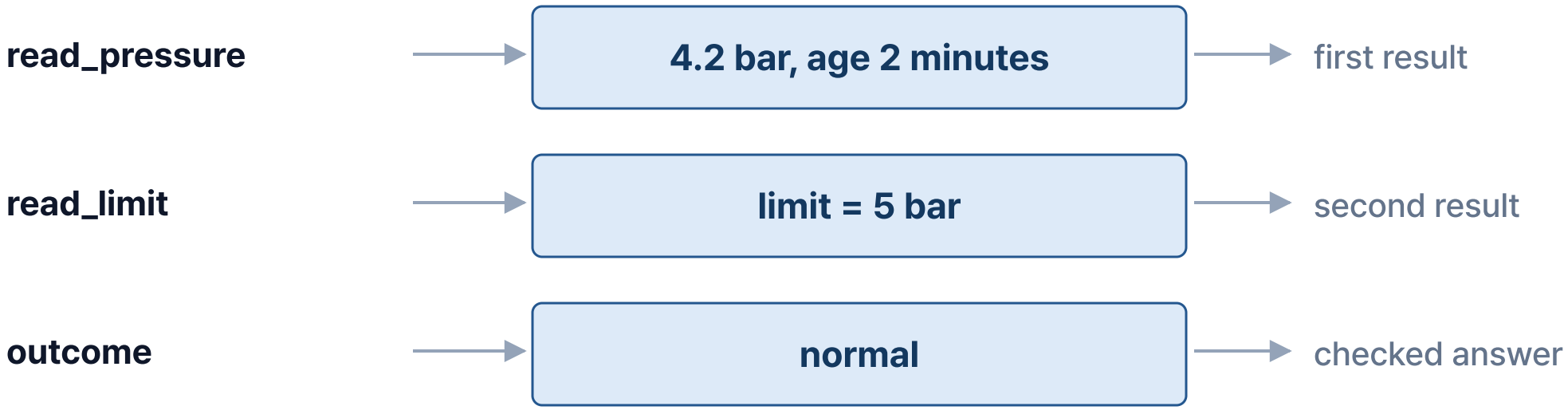
Run | Starter | Reset

Ready

LangChain.js · JavaScript · Ctrl/Cmd+Enter · Real library execution with local data

Answer: Follow the Agent Loop

The two executed results support the final outcome



The two returned observations support the recorded final response

02 · Checking and Controlling a Run

01

Choosing and Running Actions

use tool results to choose the next action

02

Checking and Controlling a Run

check evidence, enforce permissions, and bound retries

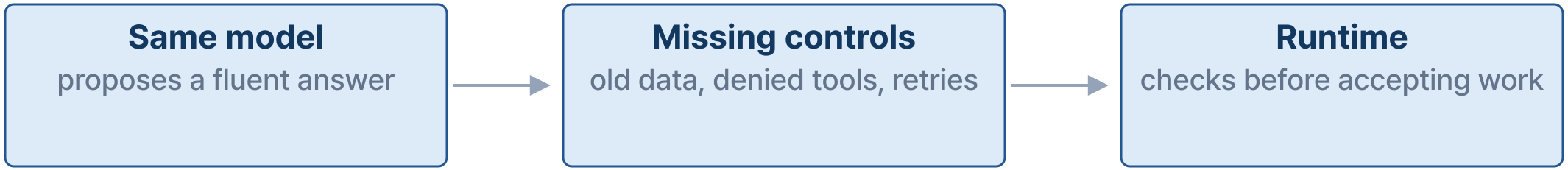
03

Connecting Steps and State

route branches and preserve progress across steps

Why Control Execution?

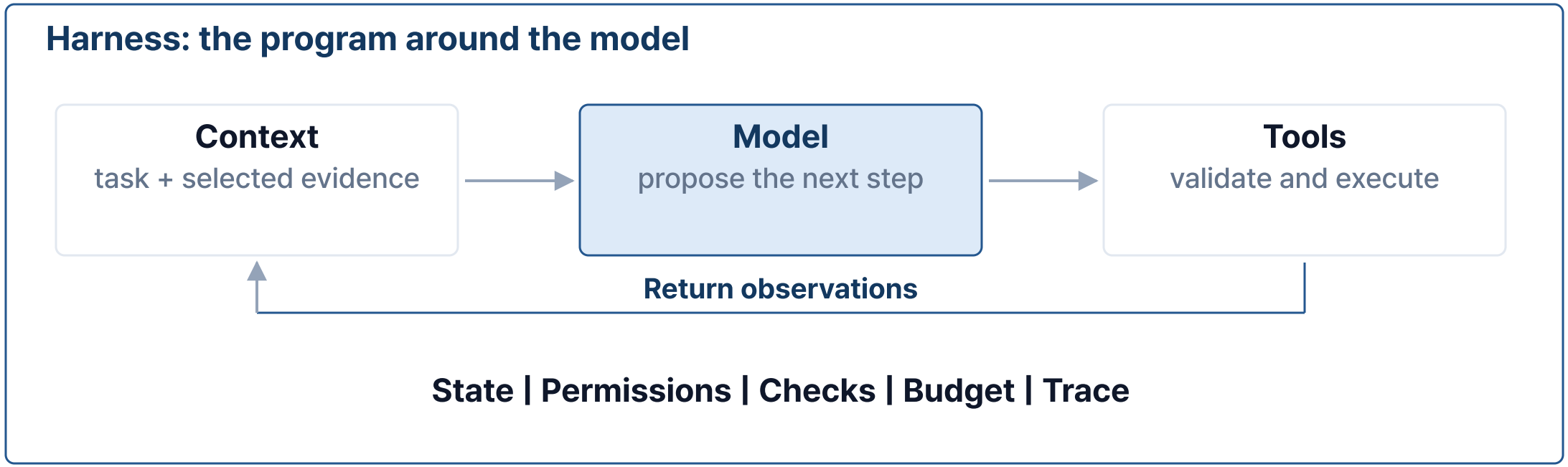
A tool loop can run without producing a valid, permitted result



Execution needs checks as well as model choices

The Agent Harness

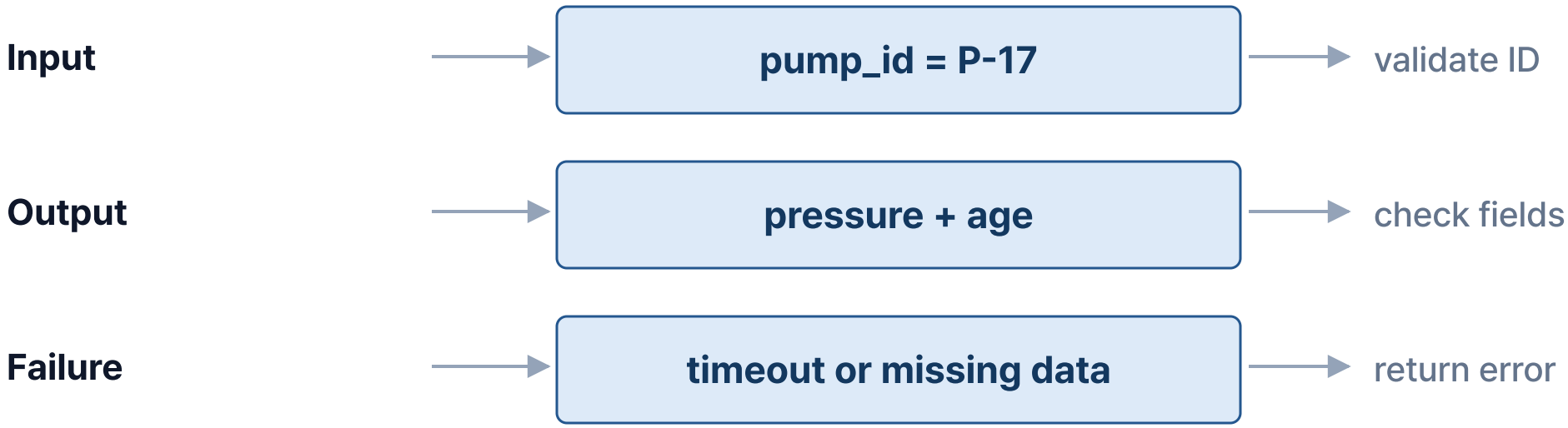
The harness supplies context, executes tools, and controls the run



The model proposes; the harness manages execution

Tool Contract

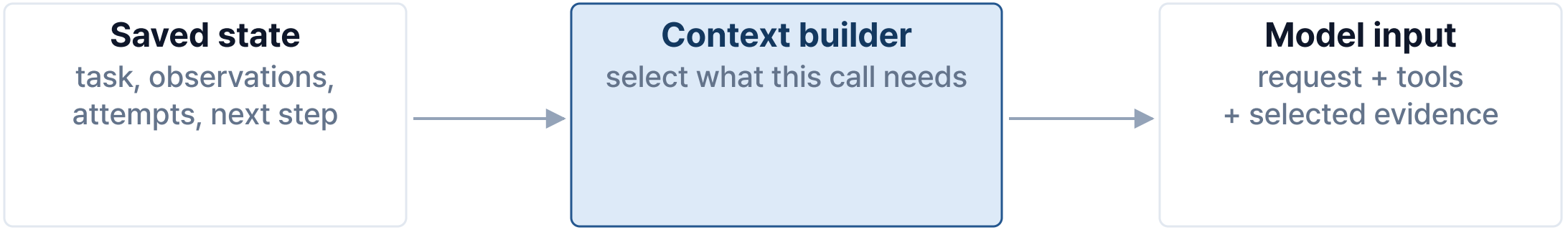
A tool contract defines inputs, outputs, and failures



A timeout supplies no pressure measurement

Context for One Call

The harness selects useful state for the next model call

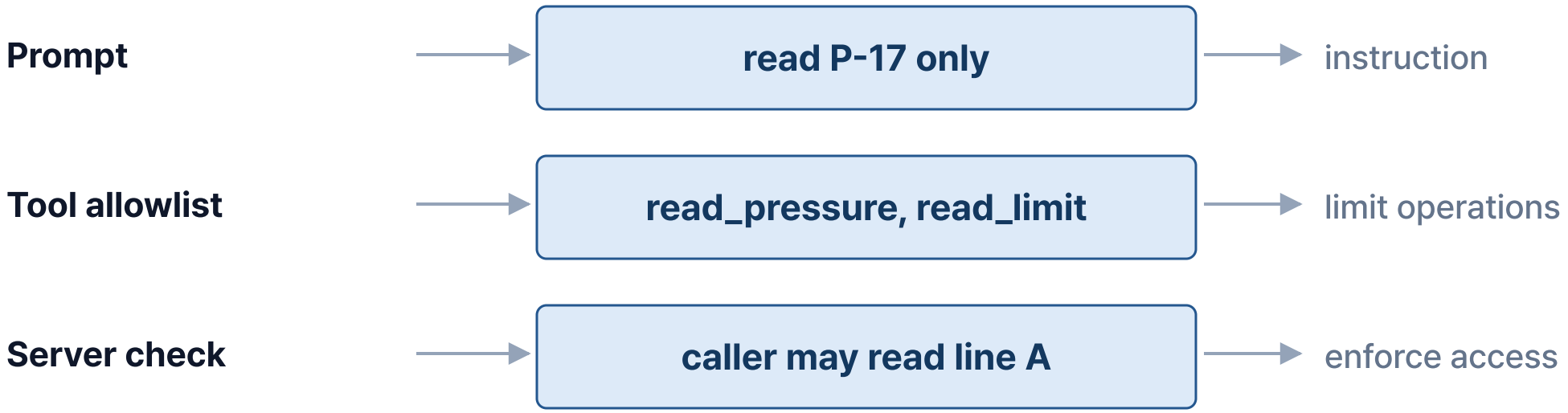


Saved state can outlive one model call

Storing a fact and sending it to the model are different operations

Permission Boundary

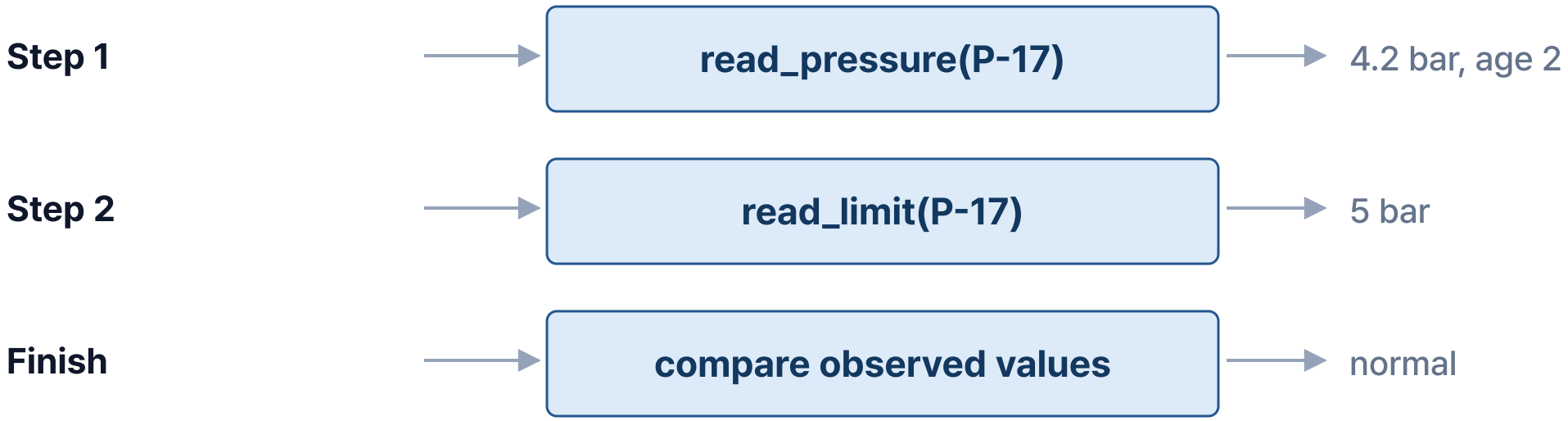
Permissions must be enforced where tools execute



An instruction alone does not grant or restrict server access

Execution Trace

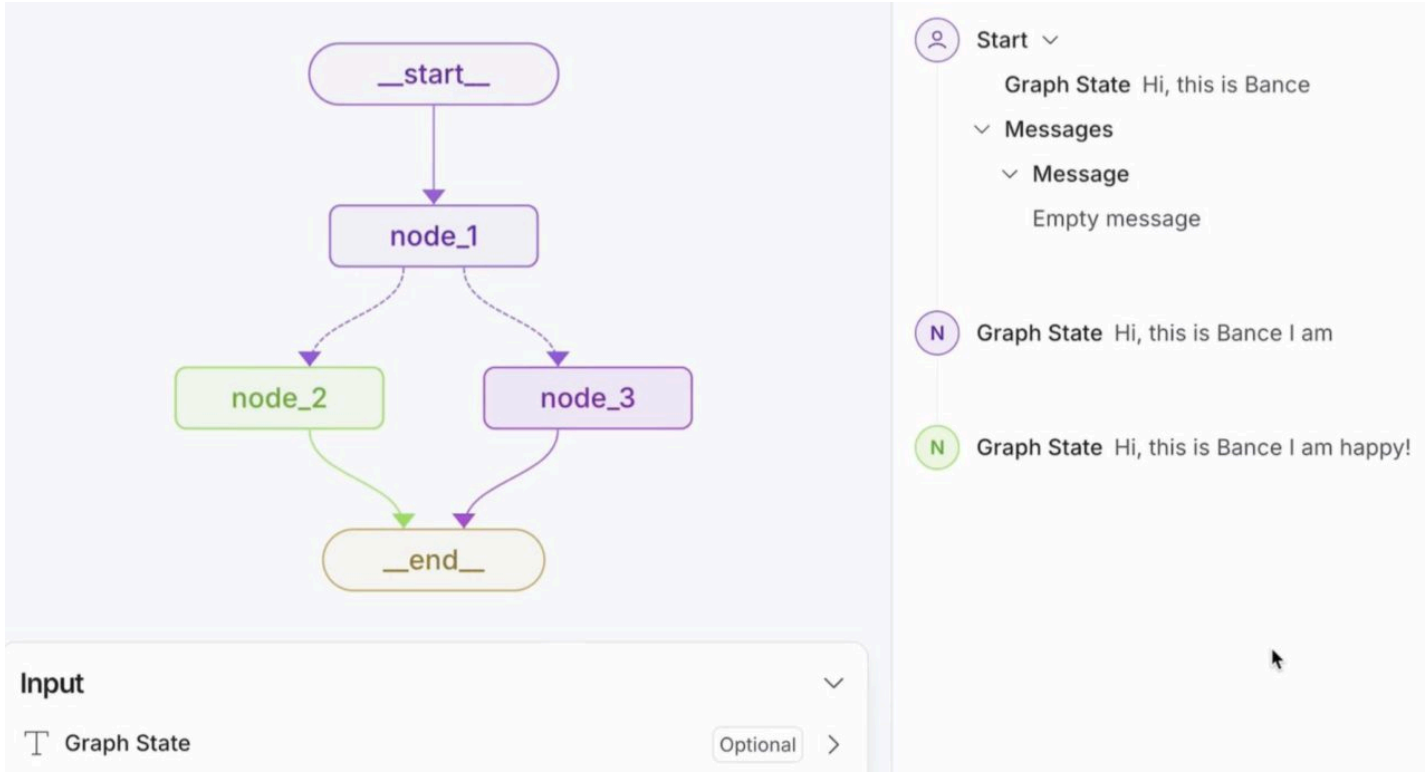
A trace records choices, inputs, results, and failures



The recorded steps explain where an answer came from

Inspecting an Execution

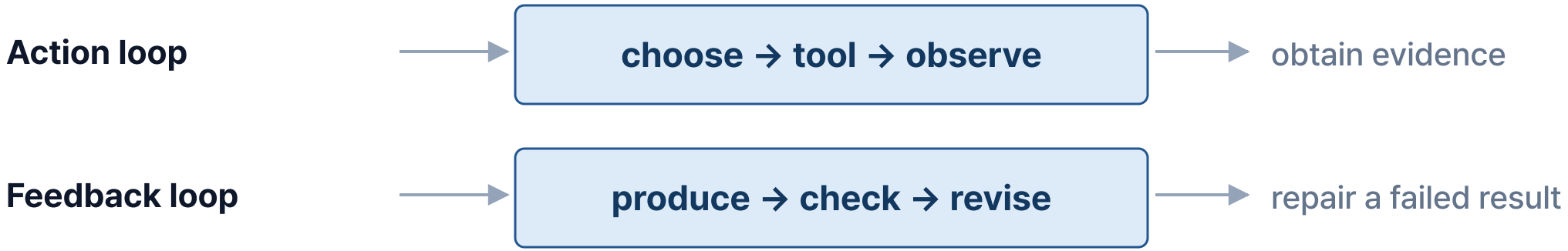
A run view connects the path taken with the messages and tool results



The graph view and message view explain different parts of the same run

Two Kinds of Loops

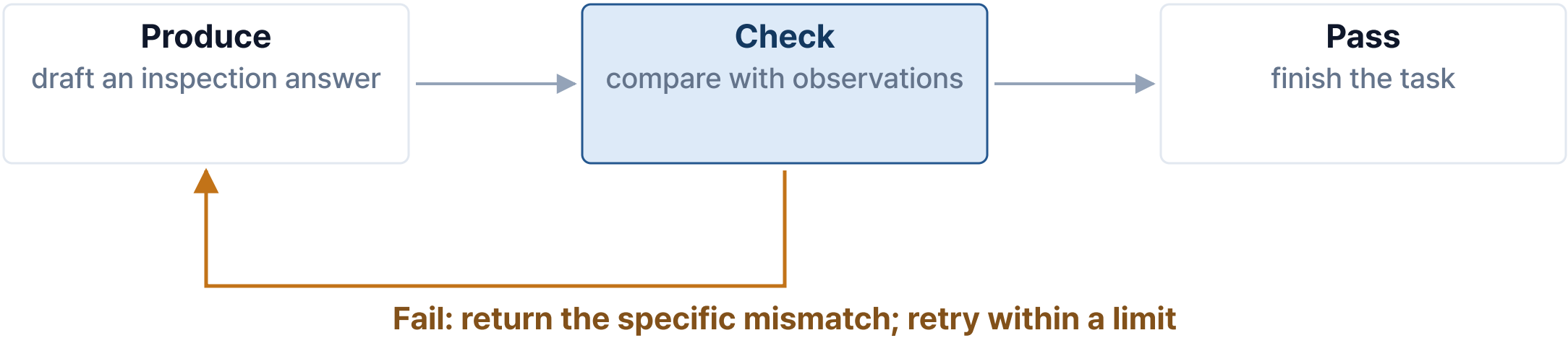
Tool use and result correction repeat for different reasons



Calling a tool twice does not establish answer quality

A Feedback Loop

A failed check returns evidence that guides the next attempt



The retry receives a reason, not just another request

Caveat: Stale Evidence

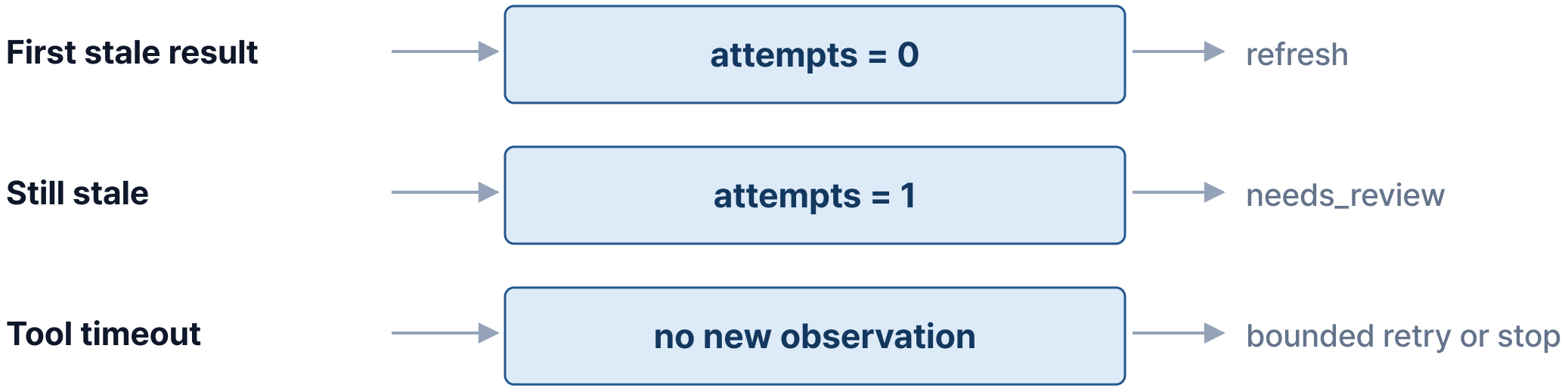
A plausible value can still fail the freshness check



The same pressure needs a different path when its age changes

Bounded Retries

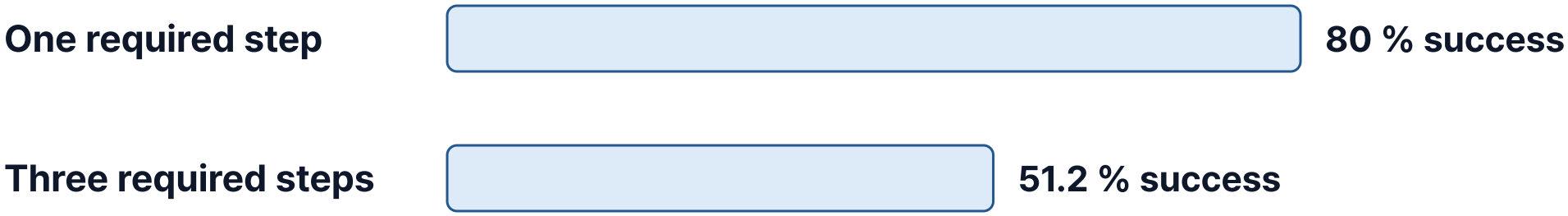
A retry policy bounds repeated work and preserves failure



A stopped run can end without completing the task

Steps and End-to-End Reliability

More steps create more opportunities for failure even when each step often succeeds



$$P(\text{all three succeed}) = 0.8 \times 0.8 \times 0.8 = 0.512$$

Three independent steps at 80% each succeed together in only 51.2% of runs

Retries Need a New Observation

Stop when retries cannot change the information needed for the decision

Observed state	Retry changes it?	Bounded action
Transient timeout	A new read may succeed	Retry within the step budget
Permission denied	Same caller remains denied	Stop; report the boundary
Freshness still too old	Repeated cached data stays old	Request refresh or review

A second identical denial supplies no new evidence for another retry

Loop Endings

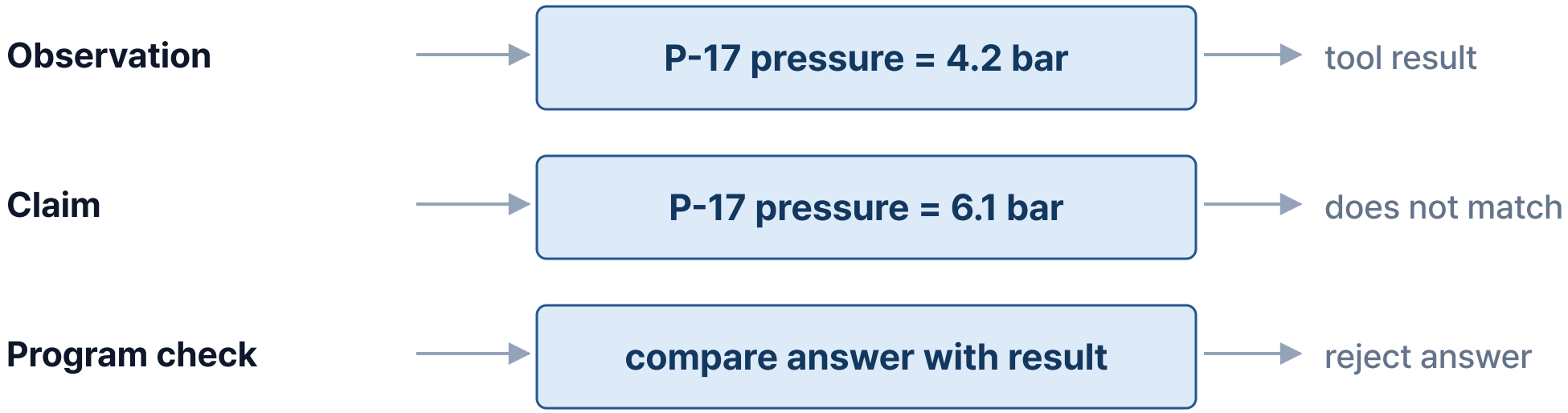
The program stops on completion, a limit, or a required decision



Reaching a limit leaves the task incomplete

Answer and Observation

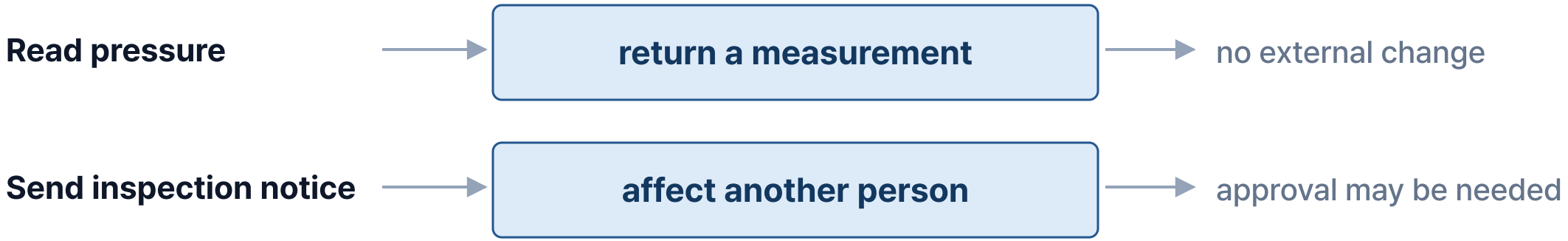
A final response still needs to match the observations



Finishing the text does not prove the task succeeded

Reading and Acting

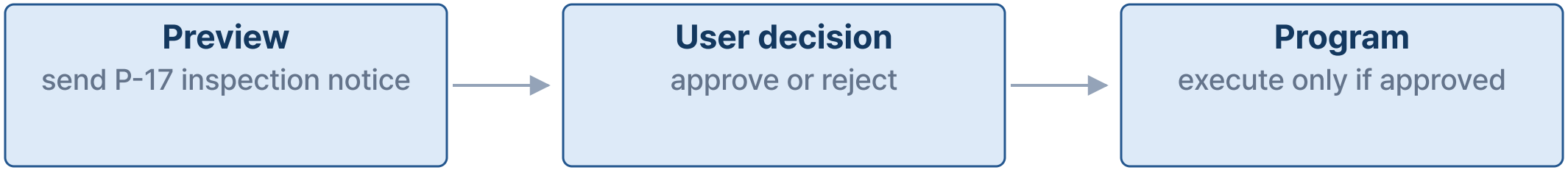
Some tools return information; others change the outside world



An action can need a decision before the program executes it

Approval Before an Action

A required approval applies to the proposed action



The program waits before the action takes effect

03 · Connecting Steps and State

01

Choosing and Running Actions

use tool results to choose the next action

02

Checking and Controlling a Run

check evidence, enforce permissions, and bound retries

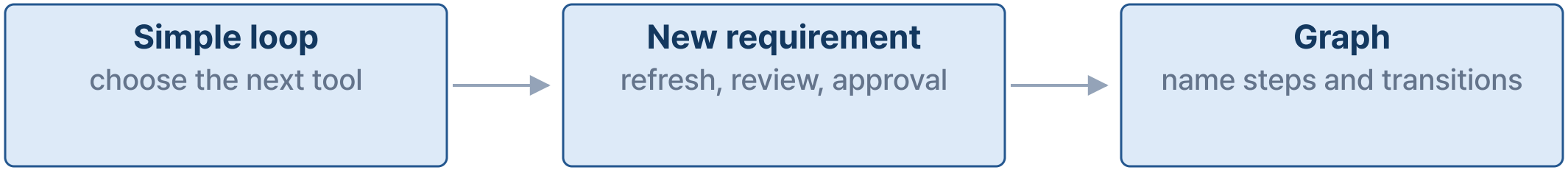
03

Connecting Steps and State

route branches and preserve progress across steps

Why an Explicit Graph?

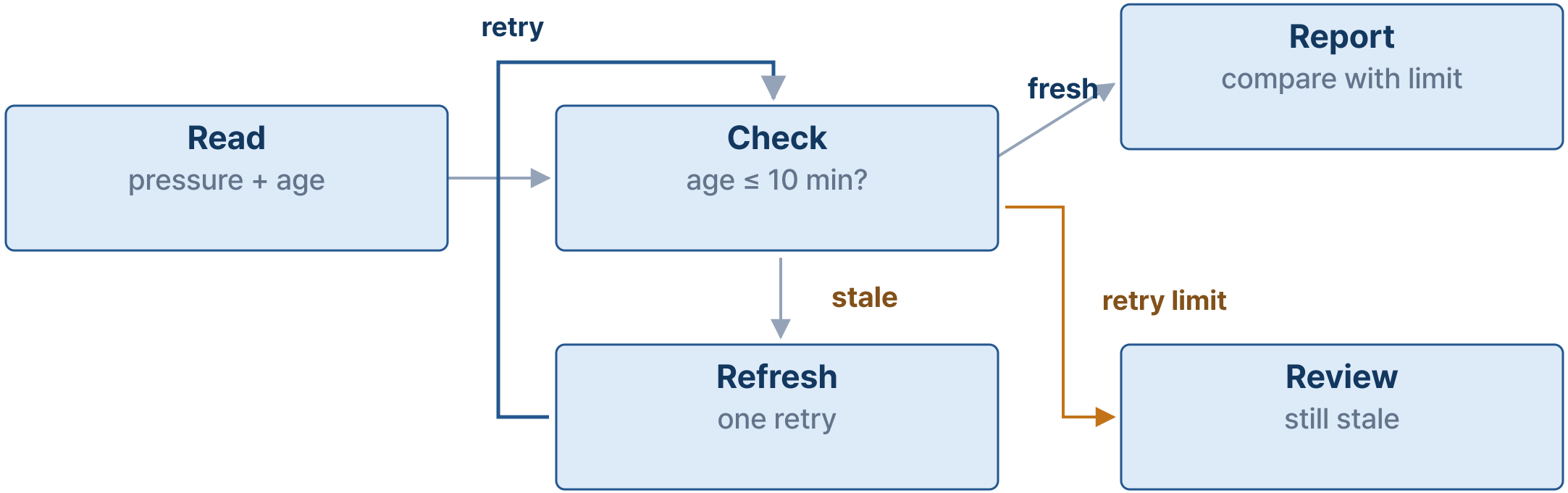
Branches and retries need an explicit execution structure



The stale reading requires a different execution path

Nodes, Edges, and State

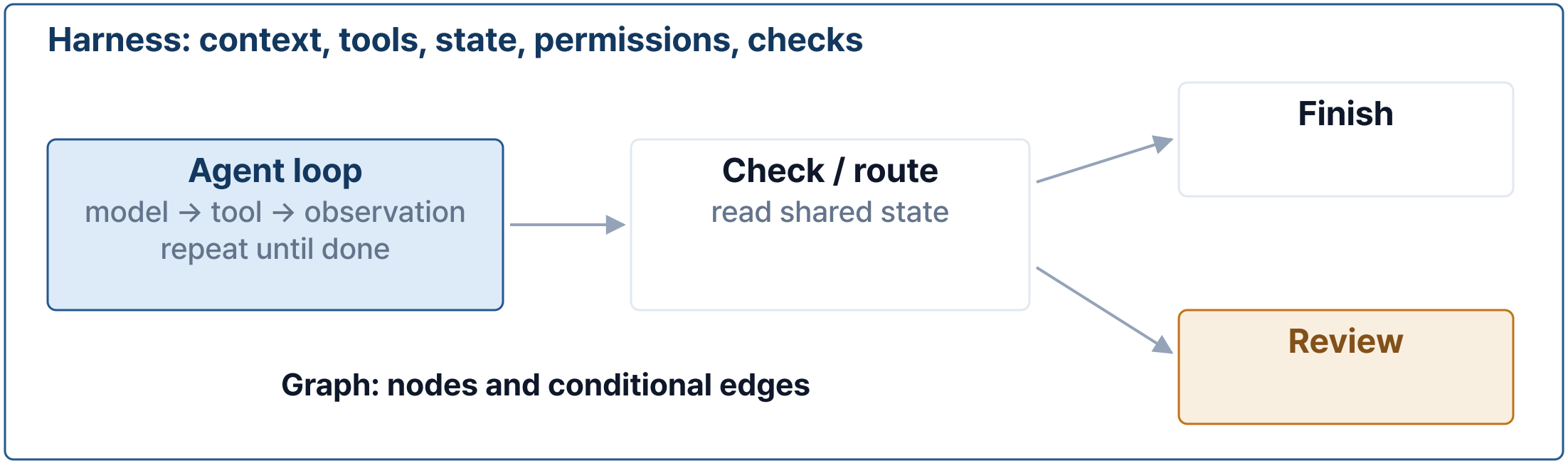
Nodes do work; edges choose the next node; state carries results



The Check node has three possible next steps

Agent, Harness, Loop, Graph

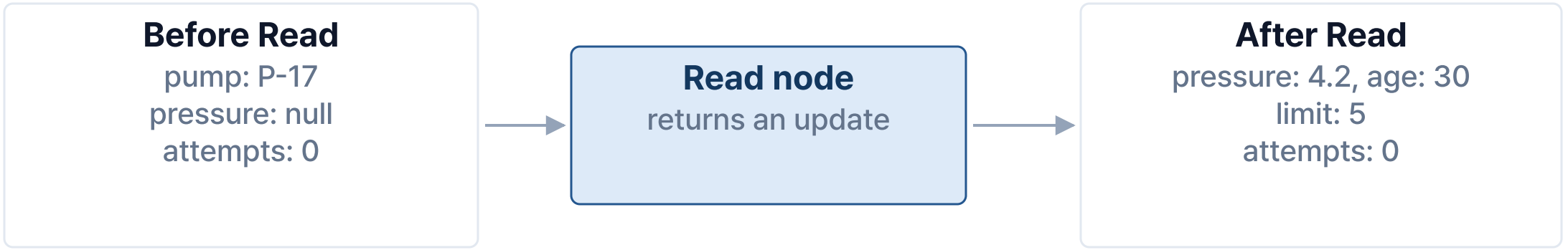
A graph can contain an agent loop, all run by a harness



The loop is inside a node; the harness surrounds execution

State Update

A node returns the fields it changed

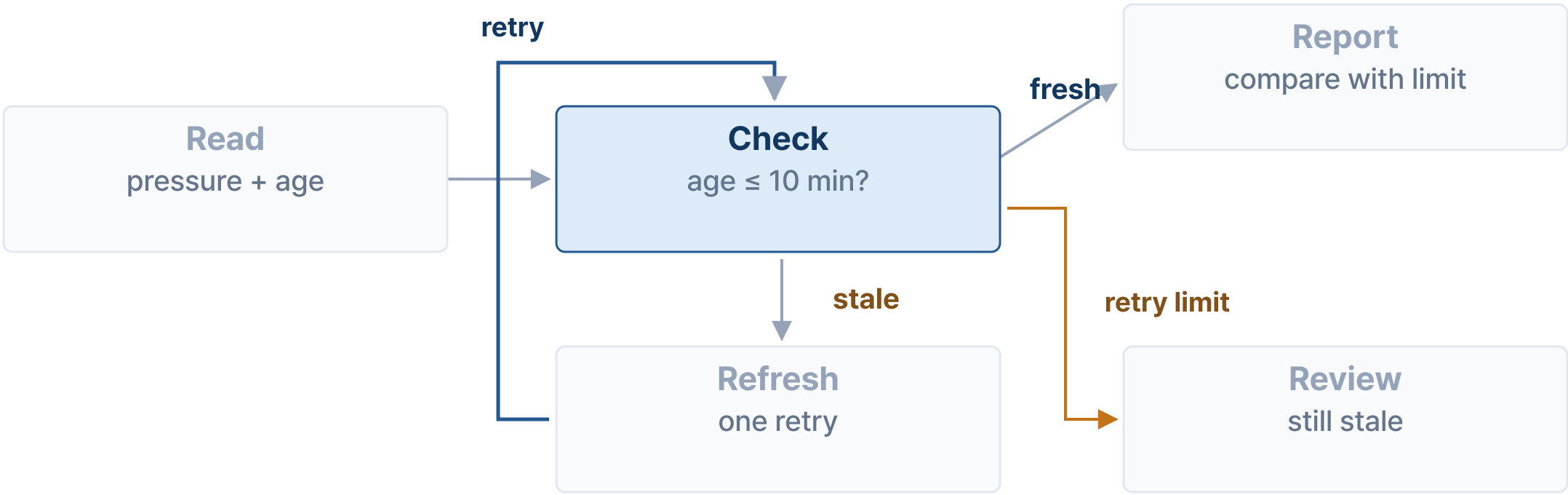


Each node reads state and returns changed fields

The Read node adds evidence to the existing task state

Conditional Routing

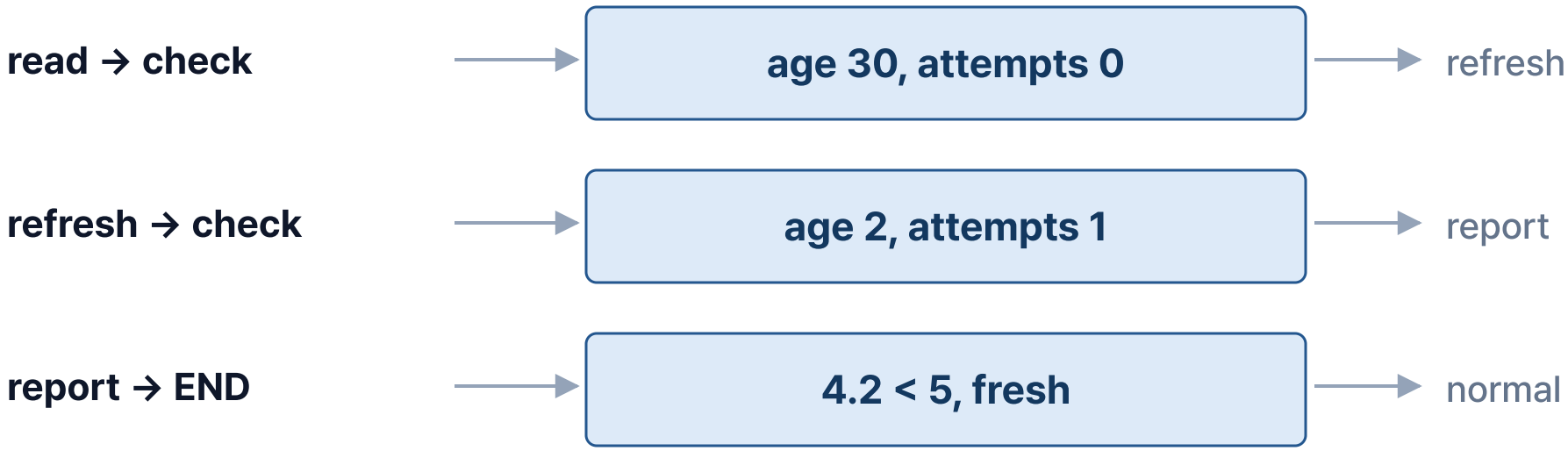
Freshness and retry count determine the next edge



Fresh → Report; stale → Refresh or Review

One Run Through the Graph

One execution follows only the edges selected by its state



A trace is one path through the graph

Demo: Run the Inspection Graph

Offline exercise: real LangGraph routing with fixed readings and no model inference

1. Run the starter and follow the trace from read to report

2. Set refreshAge = 30. Which terminal node runs?

```
Type code or click Starter
```

```
Run code to inspect its output.
```

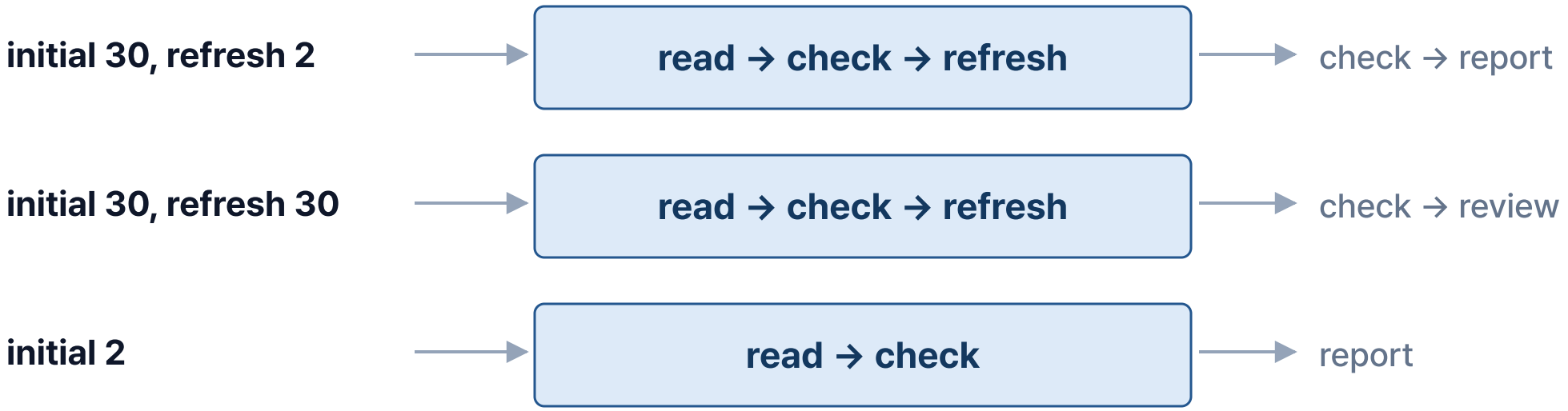
Run | Starter | Reset

Ready

LangChain.js · JavaScript · Ctrl/Cmd+Enter · Real library execution with local data

Answer: Run the Inspection Graph

The returned state determines which terminal node runs



Changing freshness changes the path, not the pressure

Parallel Reads and a Join

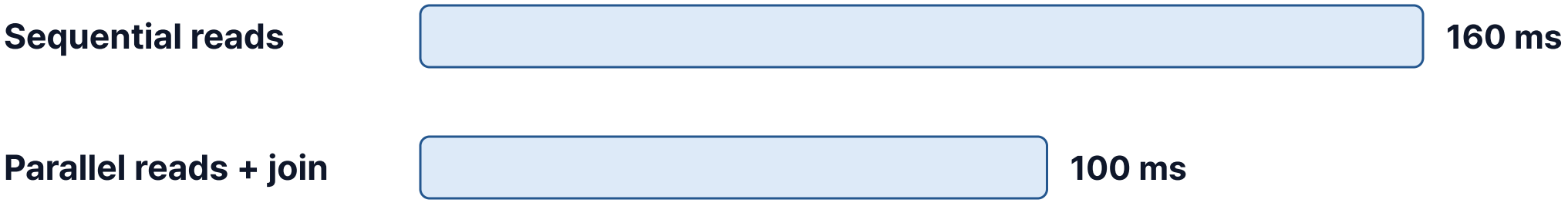
Independent reads can run together before a shared check



The comparison waits for both readings

Parallel Reads and Latency

Independent reads can overlap, but the answer must wait for both results

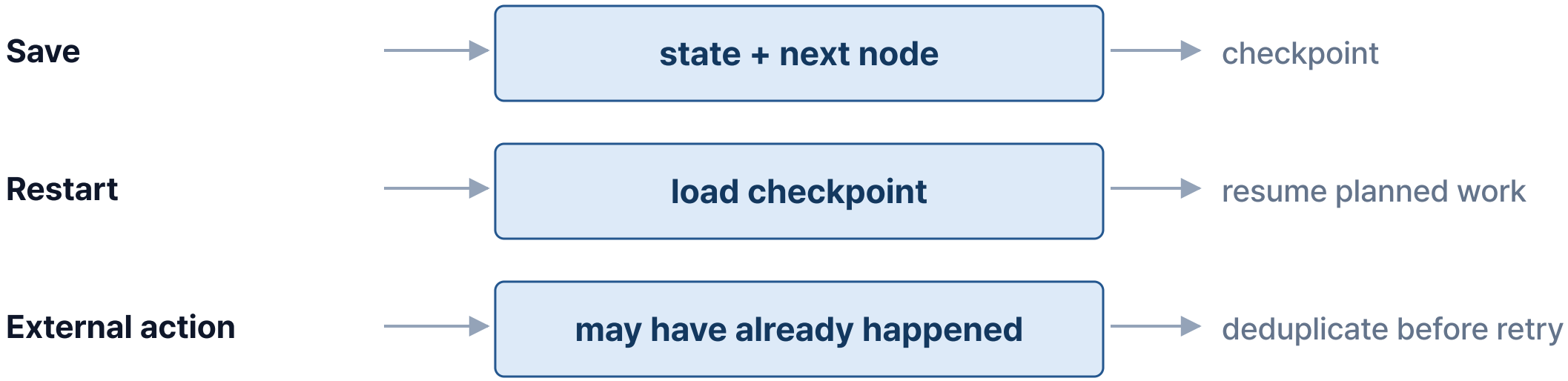



```
Sequential = 100 + 60; parallel join = max(100, 60)
```

Parallel execution waits for the slower pressure read: 100 ms

Checkpoint and Resume

A checkpoint preserves progress across process restarts



Saved state does not make external effects exactly once

Checkpoint and External Effects

Restoring graph state does not automatically undo or deduplicate a completed external write

Write completes
repair req_42 saved

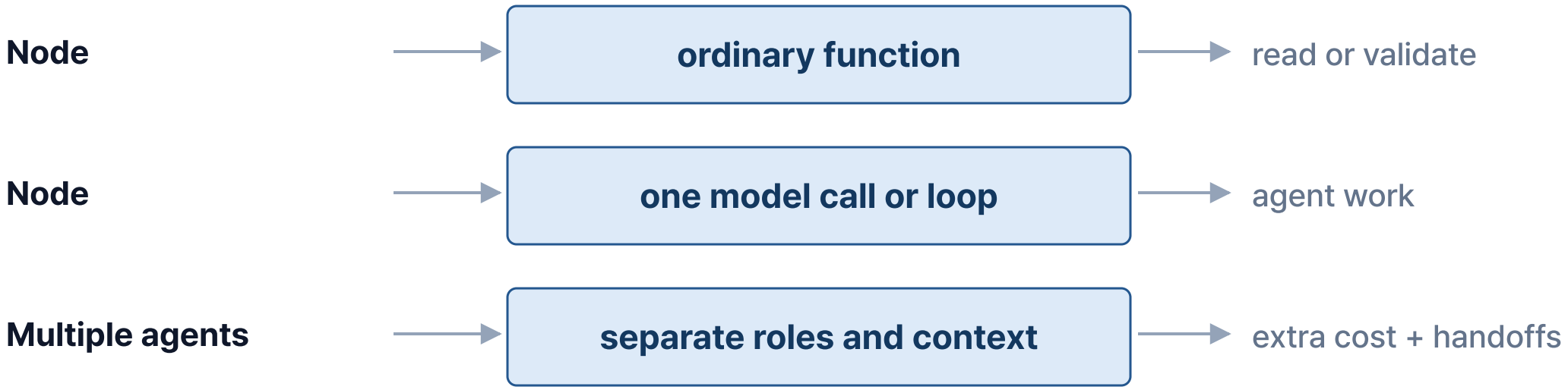
Checkpoint missing
reply or state not saved

Resume
reuse req_42; inspect receipt

The checkpoint and the external database can disagree about what already completed

Graph and Multiple Agents

A graph can connect functions, model calls, and agents



Multiple nodes do not necessarily mean multiple agents

Takeaway: Agent behavior, runtime, and control flow

1. Use observations to choose actions; keep deterministic steps explicit
2. Check contracts, permissions, evidence freshness and the budget for each run
3. Join needed results and recover external effects as well as graph state